

Formal Methods and Specification—Assignment 2

We will present the examples marked with $\searrow$ in the tutorial before you will work on the exercises.

Exercise 1

Prove the following formulas (P and Q are unary predicates, S is a 0-ary predicate, T is a predicate of arity 2, and f and g are unary function symbols):

- (a) $[\forall x . P(x)] \Rightarrow [\forall x . P(f(x))]$ ($\searrow$)
- (b) $[\forall x . P(x) \wedge Q(x)] \Rightarrow [[\forall x . P(x)] \wedge [\forall x . Q(x)]]$ (1 point)
- (c) $[\exists x . P(f(x)) \vee Q(g(x))] \Rightarrow [[\exists x . P(x)] \vee [\exists x . Q(x)]]$ (1 point)
- (d) $[\exists x . S \Rightarrow Q(x)] \Rightarrow [S \Rightarrow \exists x . Q(x)]$ (1 point)
- (e) $[\neg \exists x \exists y . T(x, y)] \Rightarrow [\forall x . \neg T(f(g(x)), f(x))]$ (1 point)

Please follow the same rules as stated at the end of Exercise 2. Especially, I will *not* tolerate any violation of the principle that proof rules can only be applied to the outer-most symbol of a formula.

You can find many example proofs in the slides of the lecture and the lecture notes, where the lecture notes contains some additional examples that I did not present during the lecture.

Exercise 2

The goal of Exercise 2d from the previous assignment sheet was to develop a specification for checking whether a given string contains a sub-string that is a palindrome of length 7. Assume that the result returns the output 1, if the given string contains such a palindrome, and 0, if not. To enable an independent check, whether this result is correct, one can extend the specification with a certificate. In the case where the answer is 1, this is trivial, using the position of the palindrome as the certificate. Describe how such a certificate looks like in the case where the answer is 0,

- (a) describing the certificate informally, using natural language, and
- (b) describing the certificate formally, using predicate logic.

It should be possible to check the certificate without nested loops. Example 2 in the chapter “Program Specification” in the lecture notes may give you some inspiration. Especially the last sentence of this example, gives an intuitive idea of how such a certificate looks like for a related string search problem.

(1+1 points)

Exercise 3

In the following, the variable name l refers to lists, a refers to arrays, i, j, k refer to non-negative integers. The variables x, y refer to array/list elements which you may assume to be integers. For all other variable names, and any constant that you use, you can assign types as you want. You can find examples in Section 2.3 of the preliminary lecture notes (the examples might give you important hints).

(a) In the theory of lists, prove

$$\forall l, x, y . l = \text{cons}(x, \text{cons}(y, \text{empty}())) \Rightarrow \text{first}(\text{rest}(l)) = y \quad (1 \text{ points})$$

(b) In the theory of lists, prove

$$\exists l . \text{rest}(l) = \text{empty}() \quad (1 \text{ points})$$

(c) In the theory of arrays, prove

$$\forall a, i, j, x . \text{write}(a, i, x)[j] = x \Rightarrow [i = j \vee a[j] = x] \quad (1.5 \text{ points})$$

(d) Prove $\forall k . 0 + k = k$ from the Peano axioms *without* the induction axiom, using induction as you learned it in school. (1.5 points)

(e) Prove $\forall k . 0 + k = k$ from the Peano axioms. (1.5 points)

In exercises (a), (b), and (c) you may use any knowledge about the integers you want, in exercises (d) and (e) *only* the Peano axioms. You are not allowed to use any equivalence rules except the following:

- commutativity of conjunction and disjunction,
- equivalence of ϕ and $\neg\neg\phi$,
- the equivalence contained in the third array axiom, and
- equivalence wrt. renaming of bound variables (e.g., $\forall x . P(x)$ is equivalent to $\forall y . P(y)$)

In addition to the examples from the lectures, you can find examples in Section 2.3 of the lecture notes.